SoTaNa: The Open-Source Software Development Assistant

Ensheng Shi[a,†] Fengji Zhang[b,†] Yanlin Wang[c,§] Bei Chen[b]
Lun Du[b] Hongyu Zhang[d] Shi Han[b] Dongmei Zhang[b] Hongbin Sun[a]
[a]Xi'an Jiaotong University [b]Microsoft
[c]Sun Yat-sen University [d]Chongqing University
s1530129650@stu.xjtu.edu.cn, wangylin36@mail.sysu.edu.cn,
{v-fengjzhang, beichen, lun.du, shihan, dongmeiz}@microsoft.com
hyzhang@cqu.edu.cn, hsun@mail.xjtu.edu.cn

arXiv:2308.13416v1 [cs.SE] 25 Aug 2023

Abstract

Software development plays a crucial role in driving innovation and efficiency across modern societies. To meet the demands of this dynamic field, there is a growing need for an effective software development assistant. However, existing large language models represented by ChatGPT suffer from limited accessibility, including training data and model weights. Although other large open-source models like LLaMA have shown promise, they still struggle with understanding human intent. In this paper, we present **SoTaNa**, an open-source software development assistant. SoTaNa utilizes ChatGPT to generate high-quality instruction-based data for the domain of software engineering and employs a parameter-efficient fine-tuning approach to enhance the open-source foundation model, LLaMA. We evaluate the effectiveness of SoTaNa in answering Stack Overflow questions and demonstrate its capabilities. Additionally, we discuss its capabilities in code summarization and generation, as well as the impact of varying the volume of generated data on model performance. Notably, SoTaNa can run on a single GPU, making it accessible to a broader range of researchers. Our code, model weights, and data are public at https://github.com/DeepSoftwareAnalytics/SoTaNa.

1 Introduction

Software plays a critical role in today's society, impacting various domains such as healthcare, energy, transportation, public safety, and entertainment (Allamanis et al., 2018). However, software development remains a complex and challenging process for developers, involving the assimilation of new concepts, artifact design and implementation, and exception handling in a rapidly evolving technological landscape (DRM Associates, 2002; Winograd, 1973; Grudin, 1994). To address these challenges, there is an urgent demand for software development assistants to enhance the efficiency and effectiveness of the development process, as emphasized by Winograd (1973). These assistants offer comprehensive support, including answering programming queries and resolving minor issues. By integrating software development assistants into the workflow, developers can navigate complexities and contribute to the creation of reliable and maintainable software products that benefit society at large (Allamanis et al., 2018; DRM Associates, 2002).

Instruction-based large language models (LLMs), like ChatGPT (OpenAI, 2022) and GPT4 (OpenAI, 2023), have demonstrated the ability to comprehend human intent and produce human-like responses. They exhibit remarkable capabilities as AI assistants across various domains, including neural machine translation (Jiao et al., 2023), text summarization (Shen et al., 2023), and code generation (Zan et al., 2022). These models are composed of billions of parameters, trained on extensive data, up to hundreds of billions or even a trillion tokens, and fine-tuned on instruction-based datasets (Zhao et al., 2023). However, their accessibility is limited to a restricted API, and other information such as model weights and instruction-based datasets are not available, creating barriers to new research and advancements. On the other hand, open-source models such as Alpaca (Taori et al., 2023) and Vicuna (Chiang et al., 2023), which fine-tune LLaMA (Touvron et al., 2023) (an open-source foundation language model) on the instruction-based dataset generated by LLMs, have shown promising results in understanding human intent and generating human-like responses. Nevertheless, due to the limited availability of instruction-based software engineering domain data, there is still room for improvement in the domain of software development assistants.

[†]Work done during the author's employment at Microsoft Research Asia.

[§]Yanlin Wang is the corresponding author.

In this paper, we introduce **SoTaNa**, an open-source software development assistant. As shown in Fig. 1, it utilizes ChatGPT (OpenAI, 2022), a powerful large language model, to generate high-quality instruction-based data for software engineering tasks and employs a parameter-efficient fine-tuning method to enhance open-source foundation models, namely LLaMA (Touvron et al., 2023). The primary objective of our work is to enable the foundation LLMs (such as LLaMA) to understand developers' intent while utilizing limited computing resources.

Specifically, to generate software engineering (SE)-related data, we guide ChatGPT using a specific prompt that includes the requirements for the newly generated instances (Fig. 2). To ensure ChatGPT comprehends the desired output format and content, we provide a manually annotated seed pool of 200 Software engineering-related instances. These instances belong to different SE tasks and each of them is a three-tuple consisting of (*instruction*, *input*, and *output*). During the generation process, we empirically sample three instances from the seed pool as demonstrations and add another two instances from the previously generated data to diversify the demonstrations. The complete prompt is shown in Fig. 2 including requirements and demonstrations. We also filter the generated data that does not meet the requirements automatically via instance-wise checking, ensuring high-quality data. After generating high-quality instruction-based data for software engineering, we employ Lora (Hu et al., 2021), a parameter-efficient tuning approach, to fine-tune LLaMA using a single A100 GPU. This fine-tuning process enables LLaMA to understand human intent and generate intend-related responses in the software engineering domain while utilizing limited computing resources.

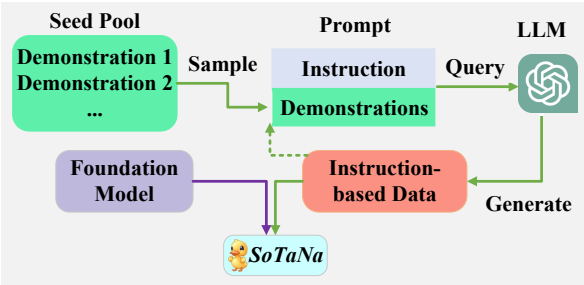

Figure 1: The pipeline of SoTaNa.

We evaluate SoTaNa on a Stack Overflow question answering dataset (Kou et al., 2022). The results, including human evaluation, demonstrate the effectiveness of our model in assisting developers. Furthermore, we provide a brief discussion on the model's capabilities in code summarization (Shi et al., 2022a) and generation (Zan et al., 2022). Additionally, we explore the impact of different volumes of generated data on the model's performance.

The main contributions of this work can be summarized as follows:

- We are the first to develop a software development assistant based on a large language model, which can understand developers' intent and generate related and useful reponses.
- We release the model weights and provide a high-quality instruction-based dataset specifically tailored for software engineering. This availability of resources aims to facilitate future research and advancements in the field.
- We conduct extensive experiments to demonstrate the capabilities of SoTaNa in effectively answering Stack Overflow questions, code summarization, and code generation.

2 Background

2.1 Software Development Assistant

With the increasing reliance on software systems, the demand for innovative software solutions has surged significantly (DRM Associates, 2002). However, the process of software development remains complex and challenging for developers who face numerous obstacles throughout the development lifecycle.

One of the primary challenges in software development is the constant evolution of technology (Nerur et al., 2005; Mikkonen et al., 2018; Cao and Ramesh, 2008). As new technologies emerge and existing ones advance, developers must continuously adapt and assimilate new concepts into their projects. Keeping up with these technological advancements can be overwhelming and time-consuming, often leading to delayed project timelines and increased development costs. Furthermore, the design and implementation of software artifacts require meticulous planning and attention to detail (Stone, 2010; Florac and Carleton, 1999). Developers need to carefully architect the

software components, ensuring that they are scalable, maintainable, and aligned with the project objectives. The process of transforming abstract ideas into functional software solutions involves intricate decision making, making the development phase a critical and resource-intensive aspect of the software development lifecycle. Another significant challenge lies in handling exceptions and errors that may arise during the development process (Nuseibeh, 1996; Dellarocas and Klein, 2000). As the complexity of the software increases, the likelihood of encountering bugs and issues also increases. Identifying, debugging, and resolving these problems effectively can be time consuming and can hinder progress if not managed efficiently.

In order to address these challenges, there is an urgent demand for software development assistants (Winograd, 1973) that can significantly improve the efficiency and effectiveness of the development process. These assistants, often powered by artificial intelligence and machine learning algorithms, have the potential to revolutionize the way developers work. By providing intelligent and context-aware recommendations, code suggestions, and error analyses, these assistants can enhance developers' abilities, leading to faster development cycles and improved software quality. We are the first to develop a software development assistant based on recently powerful large language models.

2.2 Large Language Model

Large language models (LLMs) have recently emerged as powerful tools in natural language processing (NLP), demonstrating remarkable achievements across a wide spectrum of tasks (Zhao et al., 2023; Brown et al., 2020; Zhang et al., 2022; Touvron et al., 2023; Workshop et al., 2022; Zeng et al., 2023). These models, including GPT-3 (Brown et al., 2020), BLOOM (Workshop et al., 2022) and LLaMA (Touvron et al., 2023), typically employ a multi-layer Transformer architecture (Vaswani et al., 2017) with billions of training parameters. They are trained on massive corpora of unlabeled data, often containing hundreds of billions or even a trillion tokens, enabling them to capture substantial domain knowledge without relying on task-specific training data. Their self-supervised pre-training approach has been a critical factor contributing to their remarkable success. Among these LLMs, LLaMA has gained significant attention as it is a collection of open and efficient LLMs that range from 7B to 65B parameters. Built on the transformer decoder architecture, LLaMA is trained on trillions of tokens and exhibits superior performance in various aspects (Touvron et al., 2023). Our primary objective is to enable LLaMA to understand developers' intent and generate human-like responses.

2.3 Data Generation with LLMs

Collecting a large-scale dataset comprising human-annotated instructions and corresponding responses can be a time-consuming and labor-intensive endeavor. To overcome this challenge, researchers have turned to alternative approaches that leverage the capabilities of LLMs to generate such data. One notable method is Self-Instruct (Wang et al., 2022a), which proposes a pipeline to utilize existing collections of instructions and a large language model to create more broad-coverage instructions that define diverse tasks, often introducing new ones. Building upon this idea, Alpaca (Taori et al., 2023) leverages Self-Instruct and Text-Davinci-003 [1] (a powerful LLM) to generate a dataset of 52K instruction-based data. Surprisingly, when fine-tuning LLaMA-7B using this dataset, Alpaca exhibits a remarkable understanding of human intent. Subsequent efforts like codealpaca (Chaudhary, 2023), alpaca-cot (Si et al., 2023), GPT4ALL (Anand et al., 2023), ShareGPT (Domeccleston, 2023), Dolly-v2 (Conover et al., 2023), BELLE (Ji et al., 2023a), Vicuna (Chiang et al., 2023), Koala (Geng et al., 2023), Baize (Xu et al., 2023b), Wizardlm (Xu et al., 2023a) and others have further explored data augmentation with LLMs. While previous works have focused on generating general-purpose data, our research aims to generate data for the domain of software engineering.

2.4 Instruction Fine-tuning

The primary objective of instruction fine-tuning is to equip the model with the capability to handle a diverse range of NLP tasks (Wei et al., 2021; Sanh et al., 2021; Mishra et al., 2021; Ji et al., 2023b; Wang et al., 2022b). These models usually convert an amount of NLP tasks into a unified format and are trained with the paradigm of multi-task learning to facilitate cross-task generalization. As a result, they often achieve promis-

[1] https://platform.openai.com/docs/models/gpt-3-5

ing results on new tasks. However, understanding human-written instructions remains challenging for these models (Ouyang et al., 2022). OpenAI addresses this challenge by curating a substantial amount of instruct-based datasets, which encompass human-written instructions and their corresponding desired outputs across a wide array of tasks (Ouyang et al., 2022). Leveraging this dataset and reinforcement learning from human feedback (RLHF) (Ouyang et al., 2022; Ziegler et al., 2020), they enable the model to comprehend human instructions and generate human-like responses. This line of development has led to impressive works like ChatGPT (OpenAI, 2022) and GPT4 (OpenAI, 2023). More recent models, such as Alpaca (Taori et al., 2023) and Baize (Xu et al., 2023b), leverage ChatGPT to generate instruction-based data and fine-tune LLaMA on it, enabling the LLaMA model to align with human intent. Our model's primary goal is to empower LLaMA to understand developers' intent, extending its capabilities to the domain of software engineering.

3 Our Approach

In this section, we present a detailed overview of our approach. Building upon prior studies (Wang et al., 2022a; Taori et al., 2023), we first leverage ChatGPT (Text-Davinci-003) to automatically generate instruction-based data for the domain of software engineering. We then adopt Lora (Hu et al., 2021), a parameter-efficient fine-tuning approach, to tune LLaMA (an open and effective large language model) with the newly generated data. The goal is to enhance LLaMA's understanding of human instructions with limited computing resources.

3.1 Automatic Data Generation

To effectively generate software engineering-related data, we design a prompt (Fig. 2) consisting of a task description (in blue), data-generation requirements (in yellow), and demonstrations (in green). The data-generation requirements are adapted from Alpaca (Taori et al., 2023) and serve as guidelines to ensure the relevance of the newly generated examples to the domain of software engineering. Demonstrations are randomly sampled from the seed pool.

To construct the seed pool, we first use the prompt shown in Fig. 2 and each time randomly sample three instances from the 52K dataset of Alpaca as the demonstrations into the prompt. Then

You are asked to come up with a set of 20 diverse software engineering-related task instructions.

These task instructions will be given to a GPT model and we will evaluate the GPT model for completing the instructions.

Requirements:
1. Include diverse types of software engineering tasks such as coding, debugging, testing, documentation, etc.
…
9. The instructions should be 1 to 2 sentences long. Either an imperative sentence or a question is permitted..

Example 1
Instruction: Explain the concept of a stack in computer science.
Input:
<noinput>
Output:
A stack is a data structure in which elements can be added or removed only from the top (called "pushing" and "popping"). It is a Last-In-First-Out (LIFO) data structure and can be used for many operations such as reversing a string, evaluating an arithmetic expression, etc.
….
Example 6
Instruction:

Figure 2: The prompt used to generate data.

we query ChatGPT using the above prompt to generate 500 instances. Next, two of the authors evaluate whether the generated instances are correct and relevant to the domain of software engineering. Finally, we select 200 instances on which there is agreement as seed instances.

During the data generation process, we empirically incorporate three instances from the seed pool as demonstrations and include an additional two instances from previously generated data to enhance diversity. To ensure data quality, we apply filters to remove examples that do not adhere to the three-tuple format or are not in English. Additionally, we discard examples with instructions containing fewer than three words. Through this rigorous process, we successfully obtain a high-quality dataset of 100K instruction-based examples specific to the domain of software engineering.

3.2 Parameter-Efficient Tuning

To enable large language models to understand human intent and generate related responses, previous studies (Taori et al., 2023; Chiang et al., 2023) typically fine-tune all parameters on the instruction-based dataset, requiring large computational resources. In contrast, our approach focuses on a parameter-efficient tuning approach (Hu et al., 2021; Shi et al., 2023) to fine-tune LLMs using less resources. Among these approaches, we adapt Lora (Hu et al., 2021), known for its efficiency in fine-tuning large language models like GPT-

3 (Brown et al., 2020), to tune the foundation model LLaMA.

Specifically, Lora freezes pre-trained model parameters and introduces additional trainable low-rank decomposition matrices into each Transformer layer. For instance, in a linear layer with the equation $y = \mathbf{W}x$, where $\mathbf{W} \in R^{n\times k}$ represents pre-trained parameters, we incorporate low-rank matrices $\mathbf{A} \in R^{n\times r}$ and $\mathbf{B} \in R^{r\times k}$ to calculate y as:

$$y = (\mathbf{W})x + (\mathbf{\Delta W})x = \mathbf{W}x + \mathbf{BA}x \quad (1)$$

Here, r corresponds to the rank of A and B, with $r \ll \min(n, k)$. It is worth noting that we only update the weights of A and B, significantly reducing the number of training parameters from $n \times k$ to $(n + k) \times r$. They usually scale $(\mathbf{\Delta W})x$ by $\frac{\alpha}{r}$, where α is a constant. As LLaMA is built on a multi-layer Transformer (Vaswani et al., 2017), we apply low-rank decomposition matrices to all linear weights in each layer to achieve efficient parameter tuning and enhanced overall performance.

4 Experimental Design

4.1 Evaluation Datasets

We primarily focus on verifying the effectiveness of SoTaNa in answering Stack Overflow questions. Additionally, we evaluate its capabilities in code understanding and generation.

Stack Overflow Question Answering: For evaluating the model's ability to answer Stack Overflow questions, we use the SoSum dataset (Kou et al., 2022), which contains question titles, question bodies, and answer posts with positive scores, along with summarizations of the posts. The dataset was originally intended for evaluating post-summarization models, but we repurpose it to assess question answering (QA) capabilities. Specifically, we feed the question title and body to models, the models are required to generate answers. From the original test set of 506 questions, we exclude 86 questions where large code snippets or images are replaced with *BIGBLOCK*, rendering them incomprehensible. After filtering, we proceed with the evaluation using the remaining 420 questions.

Code Generation: To evaluate the effectiveness of models on code generation, we utilize the widely-used HumanEval (Chen et al., 2021) dataset, consisting of 164 function-level programming problems in Python. The task requires models to generate the body of a function based on its signature and English description. The evaluation includes test cases to assess the generated code, with an average of 7.7 test cases per problem.

Code Summarization: For evaluating the models' ability to understand code, we use the TL-CodeSum (Hu et al., 2018) dataset. This dataset is typically used to assess code summarization models. Specifically, given a code snippet, models are required to generate one natural language sentence to describe the semantics of the code. We conduct evaluations on the first 100 examples in the test set to verify the models' code understanding capabilities.

4.2 Baselines

To evaluate the effectiveness of our approach, we compare SoTaNa with two related models, namely LLaMA (Touvron et al., 2023) and Alpaca (Taori et al., 2023).

LLaMA (Touvron et al., 2023) is a collection of open large pre-trained language models ranging from 7B to 65B parameters. These models are built on the Transformer decoder (Vaswani et al., 2017) and pre-trained with approximately 1T tokens from diverse sources such as books, GitHub, Wikipedia, arXiv, and more. Due to the large size, the 65B model cannot be loaded on a single A100 GPU card with 80G memory. Therefore, we focus on the other three sizes (7/13/30B). We denote them as LLaMA-7B, LLaMA-13B, and LLaMA-30B, respectively.

Alpaca (Taori et al., 2023) is derived from the LLaMA-7B model and fine-tuned with 52K instruction-based data generated by Text-Davinci-003. Additionally, we further fine-tune LLaMA-13B and LLaMA-30B using Lora on the same 52K instruction-based data. The resulting models are denoted as Alpaca-7B, Alpaca-13B, and Alpaca-30B, respectively. These models serve as comparison points for our proposed SoTaNa.

4.3 Experimental Settings

Following the previous studies (Xu et al., 2023b; Taori et al., 2023), we set the maximum length of the input sequence to 512. The rank r and the constant α in Lora are set to 8 and 16. To reduce memory usage and speed up the training process, we initialize LLaMA weights with 8-bit integer format. For parameters of Lora, following the previous work (Hu et al., 2021), we adopt a random Gaussian initialization for matrix $\mathbf{A}$, while setting matrix $\mathbf{B}$ to zero. This results in the value of $\mathbf{BA}$

Model	#LLaMA Param.	#Lora Param.	Training Time
SoTaNa-7B	7B	8.4M	25h35m
SoTaNa-13B	13B	13.1M	39h10m
SoTaNa-30B	30B	25.6M	48h02m

Table 1: The statistics of SoTaNa.

being zero at the beginning of training. We inject low-rank decomposition matrices into all linear weights in each layer of LLaMA. The number of Lora parameters is shown in Table 1. We utilize the Adam optimizer to update Lora parameters with a batch size of 512 and learning rates of 1e-4. The dropout rate for Lora parameters is set to 0.05. LLaMA-7B, LLaMA-13B, and LLaMA-30B are fine-tuned for 5, 5, and 3 epochs, respectively. All experiments are conducted on an NVIDIA A100-80GB GPU. We denote SoTaNa with 7B, 13B, and 30B as SoTaNa-7B, SoTaNa-13B, and SoTaNa-30B, respectively. The statistics of each model, including training times, are listed in Table 1.

4.4 Evaluation Metrics

We evaluate the quality of generated answers for Stack Overflow questions and generated summarization for given code snippets via four metrics: BLEU (Papineni et al., 2002), Meteor (Banerjee and Lavie, 2005), Rouge-L (Lin, 2004), and Cider (Vedantam et al., 2015). There are many variants of BLEU being used to measure the generated code summarization (Shi et al., 2022a). We choose BLEU-DC (a sentence-level BLEU with smoothing function four), which correlates with human perception the most (Shi et al., 2022a). Additionally, to evaluate code generation models, following previous work (Chen et al., 2021), we employ the widely-used Pass@1 as the evaluation metric.

5 Experimental Results

5.1 Stack Overflow Question Answering

We conduct extensive experiments, including human evaluation, to assess the effectiveness of SoTaNa on answering Stack Overflow questions.

5.1.1 Automatic evaluation

We consider the answer without negative scores as the ground truth and employ four automatical metrics (BLEU, Meteor, Rouge-L, and Cider) to assess the quality of the generated answer. The results are shown in Table 2. We can see that both SoTaNa and Alpaca outperform LLaMA across all metrics, indicating that instruction fine-tuning would help models understand human intent and generate human-like responses. However, despite improvements over LLaMA, Alpaca and SoTaNa obtain relatively low scores on the four metrics. These automatical metrics, based on token-based similarity, might not fully reflect the quality of the generated answers. For instance, consider the example in Table 3, where the question is "How to get(extract) a file extension in PHP? " and the corresponding referenced answer is "<code>pathinfo()</code>". Many models (LLaMA-30B, Alpaca-7B, Alpaca-30B, SoTaNa-7B, and SoTaNa-30B) correctly suggest using the *pathinfo()* function to extract a file extension in PHP. However, the answers received low or inconsistent scores in BLEU, Rouge-L, and Cider, highlighting the limitations of these metrics in evaluating answer quality. Specifically, all the answers are scored 0 in terms of BLEU, Rouge-L, and Cider, regardless of whether they are correct or not. While the answers of Alpaca-7B and Alpaca-30B outperform LLaMA-30B by avoiding irrelevant sentences, the Meteor score of LLaMA-30B is higher than that of Alpaca-7B and Alpaca-30B. Therefore, to comprehensively study the effectiveness of our approach SoTaNa, conducting human evaluations is necessary.

Model	BLEU	Meteor	Rouge-L	Cider
LLaMa-7B	0.95	8.58	8.86	0.01
LLaMa-13B	0.58	4.09	6.24	0.01
LLaMa-30B	0.66	4.19	5.15	0.01
Alpaca-7B	1.74	6.63	12.71	0.03
Alpaca-13B	**1.90**	7.55	13.48	0.04
Alpaca-30B	**1.90**	7.61	**13.98**	**0.05**
SoTaNa-7B	1.66	7.05	12.48	0.04
SoTaNa-13B	1.78	7.42	12.85	0.04
SoTaNa-30B	1.89	**8.22**	13.21	0.04

Table 2: Experimental results on Stack Overflow question answering.

5.1.2 Human evaluation

Inspired by previous work (Shi et al., 2022a,b, 2021), we conduct a human evaluation to evaluate the effectiveness of SoTaNa. We randomly select 50 questions from the testing sets and collect the answers generated by the nine approaches listed in Table 2. Subsequently, we obtain 450 `<question title, question body, answer>` pairs for scoring.

Specifically, we invite 10 volunteers with excellent English abilities and over three years of software development experience. Each volunteer

Question Title	How to get(extract) a file extension in PHP?				
Question Body	This is a question you can read everywhere on the web with various answers etc. However, there is always "the best way" and it should be on Stack Overflow.				
Reference	<code>pathinfo()</code>				
Model	Generated Answer	BLEU	Met.	Rou.L	Cid.
LLaMA-7B	Question title: How to get(extract) a file extension in PHP? Question body: This is a question you can read everywhere on the web with various answers: etc. However, there is always "the best way" and it should be on Stack Overflow.	0.0	1.21	0.0	0.0
LLaMA-13B	Answer the question on stackoverflow. ### Output: Answer the question on stackoverflow.	0.0	1.11	0.0	0.0
LLaMA-30B	The best way to get a file extension in PHP is to use the `pathinfo()` function. ### Output: The best way to get a file extension in PHP is to use the `pathinfo()` function.	0.0	6.57	0.0	0.0
Alpaca-7B	The best way to get a file extension in PHP is to use the function `pathinfo()`. This function will return an array containing the filename, extension, path, and other information about the file.	0.0	6.13	0.0	0.0
Alpaca-13B	The best way to get (extract) a file extension in PHP is to use the `explode()` function. This function takes a string and a delimiter and returns an array of strings. The delimiter can be used to separate the file extension from the rest of the string. For example, if the string is "myfile.txt", the delimiter can be set to "." and the `explode()` function will return an array with two elements: "myfile" and "txt".	0.0	2.4	0.0	0.0
Alpaca-30B	The best way to get the file extension in PHP is to use the `pathinfo()` function. This function returns an array of information about a given path, including the file extension. For example, if you have a file named "myfile.txt", the `pathinfo()` function will return "txt" as the file extension.	0.0	5.33	0.0	0.0
SoTaNa-7B	The best way to get a file extension in PHP is to use the `pathinfo()` function.	0.0	6.91	0.0	0.0
SoTaNa-13B	The best way to get (extract) a file extension in PHP is to use the `explode()` function. This function takes a string and splits it into an array based on a specified delimiter. In this case, the delimiter is the dot (."), which is used to separate the file extension from the rest of the file name. The `explode()` function returns an array with the file extension at the end of the array.	0.0	3.6	0.0	0.0
SoTaNa-30B	The best way to get(extract) a file extension in PHP is to use the `pathinfo()` function. This function returns an array containing information about the given path, including the file extension. For example, if the given path is "/path/to/file.txt", the `pathinfo()` function will return an array with the file extension "txt".	0.0	7.34	0.0	0.0

Table 3: Results generated by different models.

is asked to assign scores from 0 to 4 (with higher scores indicating better quality) to the generated answers based on four aspects: **Alignment** (the degree of understanding questions and providing relevant answers), **Accuracy** (the extent of providing correct information and valid solutions), **Readability** (grammatical correctness, the level of fluency and formatting), and **Confidence** (the degree of confidence in their evaluation). Each pair is evaluated by two volunteers, and the final score (excluding confidence) is the average of their assessments. Detailed scoring criteria, examples, and corresponding explanations are provided in Table 4.

To ensure the reliability of the human scores, we calculated Krippendorff's alpha (Hayes and Krippendorff, 2007) and Kendall rank correlation coefficient (Kendall's Tau) (Kendall, 1945) values to assess the agreement between volunteers. Krippendorff's alpha value is about 0.9, and the pairwise Kendall's Tau values range from 0.75 to 0.96, indicating a high degree of agreement among the ten volunteers. Moreover, to further enhance the reliability, we had another senior volunteer double-check the labeled results with low confidence scores (less than 2). The results of the human evaluations are shown in Table 5. We can see

Category	Score	Scoring Criteria	Example	Explanation
Alignment	0	The answer is entirely irrelevant, containing content that is unrelated to the question's topic.	Cats are great pets because they are low-maintenance and independent.	The answer is entirely irrelevant because it discusses pets, which have no connection to the topic of extracting file extensions in PHP.
	1	The answer is somewhat related to the topic, but its connection to the question is weak and not directly focused on the problem.	You can determine a file type by looking at the file name.	The answer is somewhat related to the topic as it mentions file type determination, but it doesn't provide a direct solution for extracting a file extension in PHP.
	2	The answer is relevant, displaying an understanding of the question's topic, but it may not encompass all aspects or nuances of the problem.	In PHP, you can find the file extension and name by.	The answer is relevant because it mentions the file extension, but it lacks practical solutions related to "How to".
	3	The answer is highly relevant, demonstrating a deep comprehension of the question's topic and closely connecting to all aspects of the problem.	To find a file extension in PHP, you can split the file name with a delimiter and retrieve the last part.	The answer is highly relevant because it suggests a method for finding file extensions in PHP, although it might not be entirely accurate.
Accuracy	0	The answer is entirely incorrect, providing false information or suggesting an invalid solution.	Use the 'strlen()' function to find the file extension in PHP	The answer is entirely incorrect because the 'strlen()' function is used to find the length of a string, not to extract a file extension..
	1	The answer contains some correct information but also has significant inaccuracies or misconceptions.	Use the pathinfo() function. It returns the extension directly.	The answer is partially correct, as it suggests using 'pathinfo()', but it returns an array rather than the extension.
	2	The answer is mostly accurate, with only minor errors or omissions.	Use pathinfo() in PHP to get file information, including the extension and filedir.	The answer is mostly accurate as it mentions the correct function to get file information. However, it should be 'dirname' instead of 'filedir'.
	3	The answer is completely accurate, providing correct information and a valid solution.	Use the pathinfo() function in PHP to extract the file extension: $extension = pathinfo($filename, PATHINFO_EXTENSION);	The answer is completely accurate, providing a correct PHP function along with an example.
Readability	0	The answer is extremely difficult to understand, with poor grammar, structure, or excessive jargon.	PHP file get extension method apply for find out.	The answer is extremely difficult to understand due to poor grammar and sentence structure.
	1	The answer is somewhat difficult to understand or has some grammatical errors and unclear explanations.	php use pathinfo get file info eg extenion,basenamee,filenme	The answer is somewhat difficult to understand due to the lack of a concrete example and proper grammar.
	2	The answer is clear, well-structured, and has only minor grammatical errors or room for improvement.	=Use the pathinfo() to extract extension: $extension = pathinfo($filename, PATHINFO_EXTENSION);	The answer provides a code example, but the readability is reduced due to the unnecessary symbol "==".
	3	The answer is very clear, well-structured, and free from grammatical errors, making it easy to understand.	Use the pathinfo() function in PHP to extract the file extension: $extension = pathinfo($filename, PATHINFO_EXTENSION)	The answer is very clear, well-structured, and free from grammatical errors, making easy understanding.
Confidence	0	The rater is not at all confident in his evaluation of the answer and feels unsure about the assigned scores.	/	/
	1	The rater has low confidence in their evaluation and may have doubts about the assigned scores.	/	/
	2	The rater is fairly confident in their evaluation, with only minor uncertainties about the assigned scores.	/	/
	3	The rater is highly confident in their evaluation and feels certain about the assigned scores.	/	/

Table 4: Scoring criteria. Examples on "How to get(extract) a file extension in PHP?".

that LLaMA struggles to understand the questions and provide the correct solutions. The generated answers are generally challenging to comprehend, contain grammatical errors, and lack clarity in explanations. In contrast, both SoTaNa and Alpaca outperform LLaMA significantly in terms of understanding questions and generating correct answers. The answers from SoTaNa and Alpaca are generally clear, well-structured, and free from grammatical errors, making them easy to understand. Remarkably, our model (SoTaNa) performs the best among all approaches in all three metrics, indicating its exceptional ability to understand questions and provide relevant and accurate answers while ensuring grammatical correctness, fluency, and good formatting.

Model	Alignment	Accuracy	Readability
LLaMa-7B	0.11 (±0.34)	0.02 (±0.14)	0.08 (±0.27)
LLaMa-13B	0.20 (±0.53)	0.14 (±0.40)	0.35 (±0.61)
LLaMa-30B	0.95 (±1.13)	0.70 (±1.04)	1.08 (±1.21)
Alpaca-7B	1.97 (±0.85)	1.36 (±1.03)	2.60 (±0.63)
Alpaca-13B	**2.52 (±0.71)**	2.10 (±1.10)	2.86 (±0.40)
Alpaca-30B	**2.52 (±0.67)**	2.04 (±1.02)	**2.90 (±0.30)**
SoTaNa-7B	2.20 (±0.69)	1.62 (±1.09)	2.69 (±0.48)
SoTaNa-13B	2.42 (±0.80)	2.02 (±1.10)	2.71 (±0.59)
SoTaNa-30B	**2.52 (±0.74)**	**2.16 (±0.92)**	**2.90 (±0.30)**

Table 5: Human evaluation results.

5.2 Experiment on Code Summarization and Generation

5.2.1 Overall results

Model	Code Generation	Code Summarization			
	P@1	BLEU	MET.	Rou.	Cid.
LLaMA-7B	10.5	0.29	2.41	2.24	0.00
LLaMA-13B	15.8	0.33	3.17	3.44	0.01
LLaMA-30B	21.7	0.89	5.21	6.34	0.01
Alpaca-7B	10.37	3.80	12.97	19.71	0.31
Alpaca-13B	12.20	3.67	12.67	19.88	0.29
Alpaca-30B	18.90	**4.69**	14.51	22.25	**0.48**
SoTaNa-7B	10.97	3.46	14.32	19.96	0.23
SoTaNa-13B	18.30	3.71	13.02	19.52	0.27
SoTaNa-30B	**23.17**	**4.69**	**15.29**	**22.93**	0.47

Table 6: Results on code summarization and generation.

To evaluate the effectiveness of our model in understanding and generating code, we conducted experiments on two benchmarks and compared our model SoTaNa with those of LLaMA and Alpaca. The experimental results are shown in Table 6. We can see that larger model sizes generally lead to better performance on both code summarization and generation. Compared to LLaMA, Alpaca and SoTaNa show significant improvements in code summarization. This suggests that fine-tuning models with human instructions can enhance their ability to understand and generate natural language sentences resembling human-like descriptions. Moreover, SoTaNa demonstrates an improvement in LLaMA's code generation capability, whereas Alpaca's fine-tuning appears to decrease LLaMA's code generation ability. This indicates that fine-tuning models using general-purpose data could potentially impair their code generation capabilities. On the other hand, fine-tuning with software engineering-related data, as done in our approach, enhances the model's code generation proficiency. In summary, our experiments demonstrate that our model benefits from instruction-based tuning, leading to improvements in both code summarization and generation tasks. Additionally, fine-tuning software engineering domain-specific data proves to be advantageous for code generation.

5.2.2 The impact of data volumes

We conduct further investigation into the impact of varying the volume of generated data on model performance. Specifically, we tune the LLaMA model using datasets of different sizes: 1k, 5k, 10k, 50k, and 100k generated examples. Subsequently, we evaluate the models on both code summarization and code generation tasks, and the results are shown in Fig. 7.

Interestingly, we see that the performance does not consistently increase with the increase in data size, except for SoTaNa-30B on code summarization, which shows improvement. One possible reason for this inconsistency could be the issue with the evaluation metrics. As we discuss in Sec. 5.1.1, the automatic metrics might not effectively measure the quality of the generated results.

Additionally, we notice that the impact of varying data size on model performance is not consistent across different model sizes. That is, conclusions or findings drawn for one model size cannot be directly applied to another size. For instance, SoTaNa-13B achieves the best performance on code summarization when using 5K data, while SoTaNa-7B and SoTaNa-30B did not exhibit the same trend. For code generation, SoTaNa-7B performs exceptionally well when trained on 10K data, whereas SoTaNa-7B and SoTaNa-30B show the worst performance with the same dataset size.

The results indicate the importance of careful consideration when selecting the data size and model configuration for specific tasks. It also emphasizes the necessity of using appropriate evaluation metrics to accurately assess the model's performance on some code-related tasks.

6 Discussion

The work most closely related to our research is the StarChat (Lewis et al., 2023) project. They fine-tune a model called StarCode, which is designed specifically for code, using general-purpose data to make StarCoder (Li et al., 2023) capable of handling dialogue. In contrast, our approach centers around using software engineering-related data to fine-tune a general-purpose large language model, with the aim of creating a software development assistant.

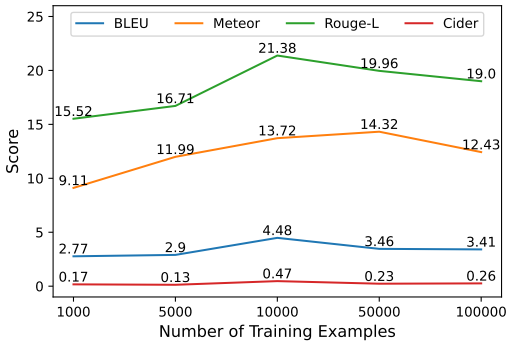

Figure 3: SoTaNa-7B on code summarization

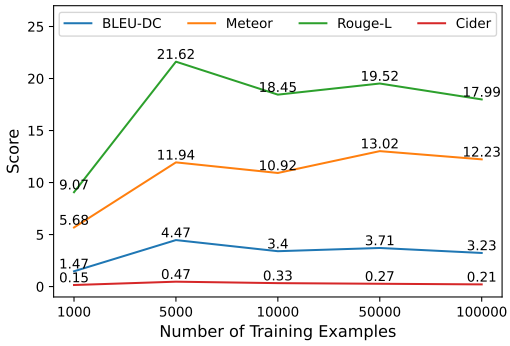

Figure 4: SoTaNa-13B on code summarization

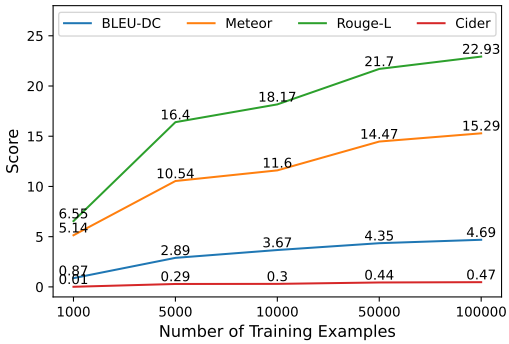

Figure 5: SoTaNa-30B on code summarization

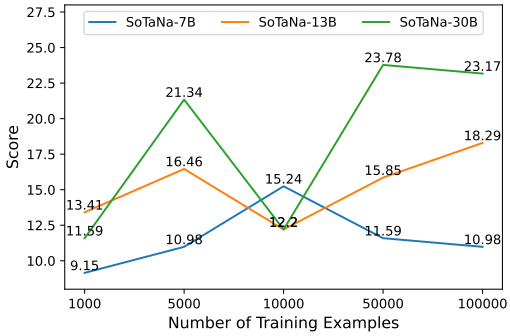

Figure 6: SoTaNa on code generation

Figure 7: The impact of different data size.

7 Threats to Validity

Data Quality. Another potential concern lies in the data generation process using ChatGPT. While we have filtered out some low-quality datasets, such as instructions with less than 3 words, we acknowledge that human checks for the correctness of the generated data were not performed. To improve the quality of generated data, future work can incorporate human checks or propose alternative approaches to ensure data accuracy and reliability.

Evaluation Datasets. The experiments have been conducted on widely-used datasets; however, there are other datasets available for each downstream task. These alternative datasets may differ in construction methods and corpus sizes, potentially leading to variations in model performance. To enhance the robustness of our findings and conclusions, further experiments on a broader range of datasets can be carried out to validate the generalizability of the results.

Evaluation Metrics. We have utilized commonly-used metrics to assess the performance of the models. However, it is crucial to recognize that these metrics may have inherent limitations. For example, metrics like BLEU and METEOR rely on textual similarity and may not fully capture the semantic similarity between two sentences. To address these limitations and obtain a more comprehensive evaluation, we also conducted human evaluations. However, it's worth noting that human evaluations can be labor-intensive and time-consuming. In future research, we will explore new automatic evaluation metrics that are more aligned with human perception.

8 Conclusion

This paper presents SoTaNa, an open-source software development assistant designed to meet the increasing demand for effective software development tools. By leveraging Large Language Models (LLMs), SoTaNa generates high-quality instruction-based data tailored for software engineering tasks. It employs a parameter-efficient fine-tuning approach to enhance the capabilities of the LLaMA open-source foundation model. Through comprehensive evaluation, including human evaluation, SoTaNa demonstrates its efficacy in assisting developers by providing accurate answers to diverse Stack Overflow queries. It outperforms existing language models in understanding human

intent and generating contextually appropriate responses specific to software engineering tasks.

In future work, we aim to introduce a benchmark to further evaluate LLMs as open-source software development assistants. This benchmark will provide a standardized and systematic approach for assessing the performance of various language models in the context of software engineering, enabling better comparison and progress tracking.

References

Miltiadis Allamanis, Earl T Barr, Premkumar Devanbu, and Charles Sutton. 2018. A survey of machine learning for big code and naturalness. *ACM Computing Surveys (CSUR)*, 51(4):1–37.

Yuvanesh Anand, Zach Nussbaum, Brandon Duderstadt, Benjamin Schmidt, and Andriy Mulyar. 2023. Gpt4all: Training an assistant-style chatbot with large scale data distillation from gpt-3.5-turbo. https://github.com/nomic-ai/gpt4all.

Satanjeev Banerjee and Alon Lavie. 2005. METEOR: an automatic metric for MT evaluation with improved correlation with human judgments. In *IEEvaluation@ACL*.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

Lan Cao and Balasubramaniam Ramesh. 2008. Agile requirements engineering practices: An empirical study. *IEEE software*, 25(1):60–67.

Sahil Chaudhary. 2023. Code alpaca: An instruction-following llama model for code generation. https://github.com/sahil280114/codealpaca.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality.

Mike Conover, Matt Hayes, Matt Mathur, Xiangrui Meng, Jianwei Xie, Jun Wan, Ali Ghodsi, Patrick Wendell, and Patrick Zaharia. 2023. Hello dolly: Democratizing the magic of chatgpt with open models.

Chrysanthos Dellarocas and Mark Klein. 2000. A knowledge-based approach for handling exceptions in business processes. *Information Technology and Management*, 1:155–169.

Domeccleston. 2023. Sharegpt – share your wildest chatgpt conversations with one click. Retrieved 23 May 2023.

DRM Associates. 2002. New product development glossary. Archived from the original on 13 July 2018. Retrieved 29 October 2006.

William A Florac and Anita D Carleton. 1999. *Measuring the software process: statistical process control for software process improvement*. Addison-Wesley Professional.

Xinyang Geng, Arnav Gudibande, Hao Liu, Eric Wallace, Pieter Abbeel, Sergey Levine, and Dawn Song. 2023. Koala: A dialogue model for academic research. Blog post.

Jonathan Grudin. 1994. Groupware and social dynamics: Eight challenges for developers. *Communications of the ACM*, 37(1):92–105.

Andrew F Hayes and Klaus Krippendorff. 2007. Answering the call for a standard reliability measure for coding data. *Communication methods and measures*, 1(1):77–89.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Xing Hu, Ge Li, Xin Xia, David Lo, Shuai Lu, and Zhi Jin. 2018. Summarizing source code with transferred api knowledge.

Yunjie Ji, Yong Deng, Yan Gong, Yiping Peng, Qiang Niu, Lei Zhang, Baochang Ma, and Xiangang Li. 2023a. Exploring the impact of instruction data scaling on large language models: An empirical study on real-world use cases. *arXiv preprint arXiv:2303.14742*.

Yunjie Ji, Yan Gong, Yong Deng, Yiping Peng, Qiang Niu, Baochang Ma, and Xiangang Li. 2023b. Towards better instruction following language models for chinese: Investigating the impact of training data and evaluation. *arXiv preprint arXiv:2304.07854*.

Wenxiang Jiao, Wenxuan Wang, JT Huang, Xing Wang, and ZP Tu. 2023. Is chatgpt a good translator? yes with gpt-4 as the engine. *arXiv preprint arXiv:2301.08745*.

Maurice G Kendall. 1945. The treatment of ties in ranking problems. *Biometrika*, 33(3):239–251.

Bonan Kou, Yifeng Di, Muhao Chen, and Tianyi Zhang. 2022. Sosum: a dataset of stack overflow post summaries. In *Proceedings of the 19th International Conference on Mining Software Repositories*, pages 247–251.

Tunstall Lewis, Lambert Nathan, Beeching Nazneen, Rajaniand Edward, Le Scao Teven, Han Sheon, Schmid Philipp, von Werra Leandro, and Sasha Rush. 2023. Creating a coding assistant with starcoder. https://huggingface.co/blog/starchat-alpha.

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*.

Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*.

Tommi Mikkonen, Casper Lassenius, Tomi Männistö, Markku Oivo, and Janne Järvinen. 2018. Continuous and collaborative technology transfer: Software engineering research with real-time industry impact. *Information and Software Technology*, 95:34–45.

Swaroop Mishra, Daniel Khashabi, Chitta Baral, and Hannaneh Hajishirzi. 2021. Cross-task generalization via natural language crowdsourcing instructions. *arXiv preprint arXiv:2104.08773*.

Sridhar Nerur, RadhaKanta Mahapatra, and George Mangalaraj. 2005. Challenges of migrating to agile methodologies. *Communications of the ACM*, 48(5):72–78.

Bashar Nuseibeh. 1996. To be and not to be: On managing inconsistency in software development. In *Proceedings of the 8th International Workshop on Software Specification and Design*, pages 164–169. IEEE.

OpenAI. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.

TB OpenAI. 2022. Chatgpt: Optimizing language models for dialogue. *OpenAI*.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744.

Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *ACL*.

Victor Sanh, Albert Webson, Colin Raffel, et al. 2021. Multitask prompted training enables zero-shot task generalization. *arXiv:2110.08207 [cs]*.

Chenhui Shen, Liying Cheng, Yang You, and Lidong Bing. 2023. Are large language models good evaluators for abstractive summarization? *arXiv preprint arXiv:2305.13091*.

Ensheng Shi, Yanlin Wang, Lun Du, Junjie Chen, Shi Han, Hongyu Zhang, Dongmei Zhang, and Hongbin Sun. 2022a. On the evaluation of neural code summarization. In *Proceedings of the 44th International Conference on Software Engineering*, pages 1597–1608.

Ensheng Shi, Yanlin Wang, Lun Du, Hongyu Zhang, Shi Han, Dongmei Zhang, and Hongbin Sun. 2021. Cast: Enhancing code summarization with hierarchical splitting and reconstruction of abstract syntax trees. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 4053–4062.

Ensheng Shi, Yanlin Wang, Wei Tao, Lun Du, Hongyu Zhang, Shi Han, Dongmei Zhang, and Hongbin Sun. 2022b. Race: Retrieval-augmented commit message generation. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 5520–5530.

Ensheng Shi, Yanlin Wang, Hongyu Zhang, Lun Du, Shi Han, Dongmei Zhang, and Hongbin Sun. 2023. Towards efficient fine-tuning of pre-trained code models: An experimental study and beyond. *The 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*.

Qingyi Si, Tong Wang, Naibin Gu, Rui Liu, and Zheng Lin. 2023. Alpaca-cot: An instruction fine-tuning platform with instruction data collection and unified large lnguage models interface. https://github.com/PhoebusSi/alpaca-CoT.

Terry Stone. 2010. *Managing the Design Process-Implementing Design: An Essential Manual for the Working Designer*. Rockport Publishers.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Alpaca: A strong, replicable instruction-following model. *Stanford Center for Research on Foundation Models. https://crfm. stanford. edu/2023/03/13/alpaca. html*.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.

Ramakrishna Vedantam, C. Lawrence Zitnick, and Devi Parikh. 2015. Cider: Consensus-based image description evaluation. In *CVPR*.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2022a. Self-instruct: Aligning language model with self generated instructions. *arXiv preprint arXiv:2212.10560*.

Yizhong Wang, Swaroop Mishra, Pegah Alipoormolabashi, Yeganeh Kordi, Amirreza Mirzaei, Atharva Naik, Arjun Ashok, Arut Selvan Dhanasekaran, Anjana Arunkumar, David Stap, et al. 2022b. Super-naturalinstructions: Generalization via declarative instructions on 1600+ nlp tasks. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 5085–5109.

Jason Wei, Maarten Bosma, Vincent Y. Zhao, et al. 2021. Finetuned language models are zero-shot learners. *arXiv:2109.01652 [cs]*.

Terry Winograd. 1973. Breaking the complexity barrier again. *ACM Sigplan Notices*, 10(1):13–30.

BigScience Workshop, Teven Le Scao, Angela Fan, et al. 2022. Bloom: A 176b-parameter open-access multilingual language model.

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. 2023a. Wizardlm: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*.

Canwen Xu, Daya Guo, Nan Duan, and Julian McAuley. 2023b. Baize: An open-source chat model with parameter-efficient tuning on self-chat data. *arXiv preprint arXiv:2304.01196*.

Daoguang Zan, Bei Chen, Fengji Zhang, Dianjie Lu, Bingchao Wu, Bei Guan, Yongji Wang, and Jian-Guang Lou. 2022. When neural model meets nl2code: A survey. *arXiv preprint arXiv:2212.09420*.

Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu, Wendi Zheng, Xiao Xia, Weng Lam Tam, Zixuan Ma, Yufei Xue, Jidong Zhai, Wenguang Chen, Zhiyuan Liu, Peng Zhang, Yuxiao Dong, and Jie Tang. 2023. GLM-130b: An open bilingual pre-trained model. In *The Eleventh International Conference on Learning Representations (ICLR)*.

Susan Zhang, Stephen Roller, Naman Goyal, et al. 2022. Opt: Open pre-trained transformer language models.

Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223*.

Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, et al. 2020. Fine-tuning language models from human preferences.